Pypomp: Inference for partially observed Markov process models in Python with JAX
DRAFT IN PROGRESS

Aaron J. Abkemeier[1] aaronabk@umich.edu
Jun Chen[2] chenjunc@umich.edu
Kevin Tan[3] kevtan@wharton.upenn.edu
Jesse Wheeler[4] jesse.wheeler@usu.edu
Bo Yang[5] ybb@umich.edu
Kunyang He[1] kunyanghe@umich.edu
Jonathan Terhorst[1] jonth@umich.edu
Aaron A. King[6] kingaa@umich.edu
Giles Hooker[3] ghooker@wharton.upenn.edu
Edward L. Ionides[1] ionides@umich.edu

[1] *University of Michigan, Department of Statistics;* [2] *Duke University, Department of Statistics;* [3] *University of Pennsylvania, Department of Statistics;* [4] *Utah State University, Department of Mathematics & Statistics;* [5] *Columbia University, Department of Marketing;* [6] *University of Michigan, Department of Ecology & Evolutionary Biology*

Editor:

Abstract

Particle filtering methods provide statistically efficient inference for nonlinear, non-Gaussian partially observed Markov process (POMP) models that arise in various areas including epidemiology, ecology, and robotics. However, the attractive inferential properties of particle filters are offset by their high computational requirement. We introduce Pypomp, a Python library for inference on POMP models that uses JAX to take advantage of GPU hardware, automatic differentiation, and just-in-time compilation to improve particle filtering estimates. Pypomp follows the structural principles of the widely used R package pomp, while obtaining considerably higher performance: Pypomp running on one GPU can obtain the throughput of 8,000 CPU cores running pomp.

Keywords: pomp, JAX, GPU acceleration, likelihood-based inference

1 Introduction

Partially observed Markov process (POMP) models provide a flexible framework for modeling dynamic systems whose underlying state is not directly observed. POMP models are also known as state-space models or hidden Markov models, though these alternative names are sometimes restricted to continuous and discrete states respectively. Pypomp is a Python library combining advances in hardware, software, and algorithms to provide novel capabilities for POMP models, with an emphasis on inference via particle filters. The particle filter (PF) (Arulampalam et al., 2002), also known as sequential Monte Carlo, provides an estimate of the likelihood of a POMP model. PF only needs to simulate from the process

model, not evaluate its transition probabilities, a property known as plug-and-play (p&p) (Bretó et al., 2009). Pypomp focuses on p&p methods because they facilitate scientific modeling, enabling applications in epidemiology (Mietchen et al., 2024; Fox et al., 2022; Wen et al., 2024), ecology (Auger-Méthé et al., 2021; Marino et al., 2019; Blackwood et al., 2013), finance (Bretó, 2014), and other domains where nonlinear mechanistic stochastic dynamic models are pertinent. Pypomp provides p&p inference methods extending PF: maximum likelihood parameter estimation by iterated filtering (Ionides et al., 2006, 2015); Bayesian inference by particle Markov chain Monte Carlo (Andrieu et al., 2010); stochastic gradient descent via the DMOP differentiable particle filter (Tan et al., 2026).

Pypomp follows the software design principles of the R package pomp (King et al., 2016), providing a general framework for specifying models and inference methods, a range of pre-built models and methods, and various graphical and diagnostic tools to assist practical data analysis. Pypomp improves on pomp by leveraging JAX (Bradbury et al., 2018) which provides just-in-time (just-in-time (JIT)) compilation, automatic differentiation (automatic differentiation (AD)), and access to massively parallel hardware such as GPUs. Other packages using AD methods for POMP models include pfjax (Lysy et al., 2022), cuthbert (Duffield, 2026), pydpf, particles, pypfilt. Pypomp distinguishes itself from these other packages by (i) an emphasis on p&p methods; (ii) following the design decisions of pomp; (iii) superior performance on our benchmark tests in Table ?????.

With enough particles and method replications to saturate it, one GPU runs iterated filtering as fast as 3,239 to 8,659 CPU cores running `pomp` or `panelPomp` (Appendix B). Such replications are routine in practice: likelihood maximization involves searching from many initial parameter vectors, and Monte Carlo adjusted profile (MCAP) confidence intervals (Ionides et al., 2017), which Pypomp computes with `mcap`, require many iterated filtering searches at each point of a likelihood profile. The GPU also makes plug-and-play inference feasible at previously intractable scales: one GPU fits a panel POMP model to the England and Wales measles data of Korevaar et al. (2020), over one million observations from 1,422 towns and cities, in 38 hours, compared with an estimated 4.8 months for `panelPomp` on a 36-core node (Section 3).

Pypomp contains advantages aside from raw GPU speed. For example, it uses AD to implement the differentiated measurement off-parameter (DMOP) particle filter, the gradient function of a differentiable particle filter, which is used for likelihood maximization by gradient descent (Tan et al., 2026). Used as a local search to refine iterated filtering estimates, it reaches higher log-likelihoods than iterated filtering alone. Pypomp also includes optimized inverse transform sampling methods as alternatives to some `jax.random` samplers, as the rejection sampling methods used by `jax.random` are inefficient on GPUs (Giles, 2016; Giles and Beentjes, 2024; Temme, 1992). On a GPU, these samplers run 23 to 44 times faster than their `jax.random` counterparts on a benchmark, and although approximate, they stay close to the exact distributions: for every parameter value tested, quantiles of their draws differ from those of a reference sampler by at most 1.1% of a standard deviation on average, as measured by the Wasserstein-1 distance (Appendix C).

When the state space is small and discrete, or the model is linear Gaussian or close enough for Gaussian filtering approximations, the particle filter is unnecessary, and the Dynamax library (Linderman et al., 2025) provides a JAX-based solution. Pypomp is aimed at more complex, nonlinear models such as those common in ecology and epidemiology,

although its plug-and-play property also makes it convenient for rapid model development even when alternative methods apply. It also supports panel POMP models and methods, including panel iterated filtering (PIF) and marginalized panel iterated filtering (MPIF) (Bretó et al., 2020; Wheeler et al., 2025), and Bayesian inference via particle Markov chain Monte Carlo (Andrieu et al., 2010) and approximate Bayesian computation (Marjoram et al., 2003).

2 POMP Modeling in Pypomp

A POMP model consists of an unobserved Markov process $\{X(t), t_0 \leq t \leq t_N\}$ and observations $Y_{1:N}$ made at times $t_{1:N}$, with observed values $y^*_{1:N}$. The initial state is defined at time t_0, $X_n := X(t_n)$, and Y_n is independent of Y_m, X_m, $m \neq n$, conditional on X_n. Plug-and-play methods need only a simulator for the initial state, $X_0 \sim f_{X_0}(x_0;\theta)$, a simulator for the state transition, $X_{n+1} \sim f_{X_{n+1}|X_n}(x_{n+1}|x_n;\theta)$, and the measurement density $f_{Y_n|X_n}(y_n|x_n;\theta)$, where θ denotes the model parameters. In Pypomp, the user passes the data and JAX-compatible programmatic descriptions of these functions to a `Pomp` object. For example, the S&P 500 model included in the package defines its state transition simulator as follows:

```
import jax
import jax.numpy as jnp
from pypomp.types import (StateDict, ParamDict, CovarDict,
                          TimeFloat, StepSizeFloat, RNGKey)

def rproc(X_: StateDict, theta_: ParamDict, key: RNGKey,
          covars: CovarDict, t: TimeFloat, dt: StepSizeFloat):
    V, S = X_["V"], X_["S"]
    mu, kappa, theta = theta_["mu"], theta_["kappa"], theta_["theta"]
    xi, rho, y_prev = theta_["xi"], theta_["rho"], covars["y_prev"]
    dZ = jax.random.normal(key)
    dWs = (y_prev - mu + 0.5 * V) / jnp.sqrt(V)
    dWv = rho * dWs + jnp.sqrt(1 - rho**2) * dZ
    S = S + S * (mu + jnp.sqrt(jnp.maximum(V, 1e-32)) * dWs)
    V = V + kappa * (theta - V) + xi * jnp.sqrt(V) * dWv
    V = jnp.maximum(V, 1e-32)
    return {"V": V, "S": S}
```

Pypomp vectorizes such a function over particles and over replicate runs, such as when starting searches from many values of θ, and JIT compiles it so that the computation runs in parallel across GPU cores.

Given a `Pomp` object, one can run methods such as the particle filter (`pfilter`) for likelihood evaluation and state estimation, iterated filtering 2 (IF2) (`mif`) for global maximum likelihood estimation, and the differentiable particle filter MOP-α (`mop`, Tan et al. (2026)), whose gradients the `train` method uses for local optimization. The following example searches globally for the maximum likelihood estimate with IF2, refines the estimate by gradient descent, and evaluates the likelihood of the result:

```
import pypomp as pp
spx_obj = pp.models.spx()
key = jax.random.key(1)
rw_sd = pp.RWSigma( # Random walk standard deviations for IF2
    sigmas={"mu": 0.02, "kappa": 0.02, "theta": 0.02,
            "xi": 0.02, "rho": 0.02, "V_0": 0.1}, init_names=["V_0"],
).geometric_cooling(a=0.5)
spx_obj.mif(rw_sd=rw_sd, J=1000, M=10, key=key) # IF2
eta = pp.LearningRate(
  {name: 0.001 for name in spx_obj.canonical_param_names}
).geometric_decay(decay_rate=0.5, M=10) # Learning rates
spx_obj.train(J=1000, M=10, eta=eta, optimizer=pp.Adam()) # Gradient descent
spx_obj.pfilter(J=1000, reps=10) # Particle filter
```

Each method call stores its log-likelihood estimates, parameter traces, execution time, and arguments in `spx_obj.results_history`, and helpers such as `traces` return these in convenient formats such as a data frame. The JIT-compiled functions behind these methods are available in `pypomp.functional` for building custom inference procedures. The source code is available at `https://github.com/pypomp/pypomp`, documentation at `https://pypomp.readthedocs.io`, and tutorials that build complete models at `https://pypomp.github.io/tutorials`. The quantitative tests that validate the correctness and speed of Pypomp's main methods are available at `https://github.com/pypomp/quant`.

3 Performance

We benchmarked Pypomp 1.0.5--1.0.8 against `pomp` 6.3 and `panelPomp` 1.7.0.0 on four models using the quantitative test suite (commit f616c89). Below, R refers to `pomp`, or `panelPomp` for the panel model, running in R 4.4.0 on a 36-core CPU node. The models differ in the cost of the process model, the number of process model steps between resampling, and whether they are panels, so together they show where the GPU helps most and least (Appendix A). Table 1 compares IF2 wall-clock times for the same workloads with R and with Pypomp on one RTX PRO 6000 Blackwell Server Edition GPU. These workloads are sized for R, and they keep only 36,000 to 180,000 particles in flight, too few to saturate the GPU. In scaling tests that add more searches or more particles per search (Appendix B), throughput keeps rising until 1.0 to 2 million particles are in flight. At that point, one GPU matches 3,239 to 8,659 CPU cores running R, the Peak column of Table 1. The S&P 500 model gains the most from scaling because its benchmark, with 1,000 particles per search and a cheap process model, gives the GPU the least work. The GPU times for the measles models rely on the approximate samplers in `pypomp.random`: with the standard `jax.random` samplers, IF2 for the London measles model took 46 times longer (Appendix C). Fitting the measles model of He et al. (2010) to all 1,422 towns and cities for 1950–1963 took one GPU 38 hours, including likelihood evaluation, versus an estimated 4.8 months for R on a 36-core node, extrapolating linearly from `panelPomp`'s cost on the 4-town panel (Appendix D).

Table 1: Wall-clock time in seconds for IF2, running 36 searches from the same starting points, with the workloads in Appendix A. *`panelPomp` for the 4-town panel. Ratio is the `pomp` time divided by the Pypomp time, and CPU cores matched is that ratio times 36. Peak cores matched is the highest throughput in terms of CPU cores matched in GPU scaling tests with more particles in flight (Appendix B).

Model	`pomp`* (36 cores)	Pypomp (1 GPU)	Ratio	CPU cores matched	Peak cores matched
Cholera, Dhaka	1,609	28.3	57×	2,045	3,239
Measles, London	2,923	32.2	91×	3,264	5,583
Measles, 4-town panel	8,262	98.2	84×	3,030	3,947
S&P 500	703	34.8	20×	727	8,659

4 Conclusion

The computational cost of particle filtering has limited both the size of datasets that practitioners can fit and how many models and likelihood profiles they can afford to evaluate. We expect Pypomp's speed will allow practitioners to iterate more quickly on model structure, fit models to datasets as large as the England and Wales measles data, and develop and evaluate more realistic models of the systems they study.

References

Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle Markov chain Monte Carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(3):269–342, 2010. doi: 10.1111/j.1467-9868.2009.00736.x.

M. Sanjeev Arulampalam, Simon Maskell, Neil Gordon, and Tim Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, 2002. doi: 10.1109/78.978374.

Marie Auger-Méthé, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, Joanna Mills Flemming, Anders Nielsen, Giovanni Petris, and Len Thomas. A guide to state-space modeling of ecological time series. *Ecological Monographs*, 91(4):e01470, 2021. doi: 10.1002/ecm.1470.

Julie C. Blackwood, Daniel G. Streicker, Sonia Altizer, and Pejman Rohani. Resolving the roles of immunity, pathogenesis, and immigration for rabies persistence in vampire bats. *Proceedings of the National Academy of Sciences of the United States of America*, 110 (51):20837–20842, 2013. doi: 10.1073/pnas.1308817110.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, ake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018.

Carles Bretó. On idiosyncratic stochasticity of financial leverage effects. *Statistics & Probability Letters*, 91:20–26, 2014. doi: 10.1016/j.spl.2014.04.003.

Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, 2009. doi: 10.1214/08-AOAS201.

Carles Bretó, Edward L. Ionides, and Aaron A. King. Panel Data Analysis via Mechanistic Models. *Journal of the American Statistical Association*, 115(531):1178–1188, 2020. doi: 10.1080/01621459.2019.1604367.

Sam Duffield. cuthbert: State-space model inference with JAX, 2026. URL `https://pypi.org/project/cuthbert/`.

Spencer J. Fox, Michael Lachmann, Mauricio Tec, Remy Pasco, Spencer Woody, Zhanwei Du, Xutong Wang, Tanyi A. Ingle, E. Javan, Maytal Dahan, Kelly Gaither, Mark E. Escott, Stephen I. Adler, S. Claiborne Johnston, James G. Scott, and Lauren A. Meyers. Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119, 2022. doi: 10.1073/pnas.2111870119.

Michael B. Giles. Algorithm 955: Approximation of the Inverse Poisson Cumulative Distribution Function. *ACM Transactions on Mathematical Software*, 42(1):1–22, 2016. doi: 10.1145/2699466.

Michael B. Giles and Casper Beentjes. Approximation of an inverse of the incomplete Beta Function. In Kevin Buzzard, Alicia Dickenstein, Bettina Eick, Anton Leykin, and Yue Ren, editors, *Mathematical Software – ICMS 2024*, volume 14749, pages 207–214. Springer Nature Switzerland, Cham, 2024. doi: 10.1007/978-3-031-64529-7_22. Series Title: Lecture Notes in Computer Science.

Daihai He, Edward L. Ionides, and Aaron A. King. Plug-and-play inference for disease dynamics: Measles in large and small populations as a case study. *Journal of The Royal Society Interface*, 7(43):271–283, 2010. doi: 10.1098/rsif.2009.0151.

Edward L. Ionides, Carles Breto, and Aaron A. King. Inference for nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 103(49):18438–18443, 2006. doi: 10.1073/pnas.0603181103.

Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, 2015. doi: 10.1073/pnas.1410597112.

Edward L. Ionides, Carles Breto, Joonha Park, R. Alexander Smith, and Aaron A. King. Monte Carlo profile confidence intervals for dynamic systems. *Journal of The Royal Society Interface*, 14(132):20170126, 2017. doi: 10.1098/rsif.2017.0126.

Aaron A. King, Edward L. Ionides, Mercedes Pascual, and Menno J. Bouma. Inapparent infections and cholera dynamics. *Nature*, 454(7206):877–880, 2008. doi: 10.1038/nature07084.

Aaron A. King, Dao Nguyen, and Edward L. Ionides. Statistical Inference for Partially Observed Markov Processes via the *R* Package **pomp**. *Journal of Statistical Software*, 69(12), 2016. doi: 10.18637/jss.v069.i12.

Hannah Korevaar, C. Jessica Metcalf, and Bryan T. Grenfell. Structure, space and size: Competing drivers of variation in urban and rural measles transmission. *Journal of The Royal Society Interface*, 17(168):20200010, 2020. doi: 10.1098/rsif.2020.0010.

Scott W. Linderman, Peter Chang, Giles Harper-Donnelly, Aleyna Kara, Xinglong Li, Gerardo Duran-Martin, and Kevin Murphy. Dynamax: A python package for probabilistic state space modeling with JAX. *Journal of Open Source Software*, 10(108):7069, 2025. doi: 10.21105/joss.07069.

Martin Lysy, Pranav Subramani, Jonathan Ramkissoon, Mohan Wu, Michelle Ko, and Kanika Choptra. Pfjax: Particle filtering in JAX, 2022. URL `https://github.com/mlysy/pfjax`.

John A. Jr. Marino, Scott D. Peacor, David B. Bunnell, Henry A. Vanderploeg, Steven A. Pothoven, Ashley K. Elgin, James R. Bence, Jing Jiao, and Edward L. Ionides. Evaluating consumptive and nonconsumptive predator effects on prey density using field time-series data. *Ecology*, 100(3):e02583, 2019. doi: 10.1002/ecy.2583.

Paul Marjoram, John Molitor, Vincent Plagnol, and Simon Tavaré. Markov chain Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences*, 100(26): 15324–15328, 2003. doi: 10.1073/pnas.0306899100.

Matthew S. Mietchen, Erin Clancey, Corrin McMichael, and Eric T. Lofgren. Estimating SARS-CoV-2 transmission parameters between coinciding outbreaks in a university population and the surrounding community, 2024.

Kevin Tan, Aaron J. Abkemeier, Kunyang He, Giles Hooker, and Edward L. Ionides. Automatic differentiation for inference on partially observed Markov process models. *arXiv:2407.03085v2*, 2026. doi: 10.48550/arXiv.2407.03085.

Nico M. Temme. Asymptotic inversion of incomplete gamma functions. *Mathematics of Computation*, 58(198):755–764, 1992. doi: 10.1090/S0025-5718-1992-1122079-8.

Li Wen, Yi Yin, Qiong Li, Zhihang Peng, and Daihai He. Modeling the co-circulation of influenza and COVID-19 in Hong Kong, China. *Advances in Continuous and Discrete Models*, 2024(32(2024)):1–9, 2024. doi: 10.1186/s13662-024-03830-7.

Jesse Wheeler, Aaron J. Abkemeier, and Edward L. Ionides. Iterating marginalized Bayes maps for likelihood maximization with application to nonlinear panel models. *arXiv:2511.17438*, 2025. doi: 10.48550/arXiv.2511.17438.

Appendix A. Full benchmark results

The four benchmark models stress different parts of the particle filter, which helps locate performance bottlenecks.

Cholera, Dhaka. The cholera model of King et al. (2008) has 600 monthly observations with 20 Euler steps between each, and each step updates a stochastic differential equation with one normal draw. The process model is cheap, but there are many steps between resamples, so this model isolates the per-step overhead inside the process model.

Measles, London. The measles model of He et al. (2010) has weekly case reports from 1950 to 1963 and takes a daily Euler step, interpolating covariates and drawing binomial, gamma, and Poisson random variables at every step. It has the most expensive process model of the four, so it isolates how the GPU handles heavy computation per particle, and it relies on the approximate samplers in `pypomp.random` (Appendix C).

Measles, 4-town panel. The panel model fits the same measles model to London, Halesworth, Hastings, and Cardiff jointly, with some parameters shared across towns and the rest specific to each town. MPIF (Wheeler et al., 2025) filters the towns sequentially within each iteration, so an iteration costs about four single-town iterations with the same number of particles in flight. This model tests the panel methods used in Appendix D. Its ratio of R time to GPU time is lower than for the London model because R runs it relatively fast, likely because the random variate generators in R can take a cheaper route for the smaller parameter values of the smaller towns.

S&P 500. The stochastic volatility model of Bretó (2014) has 3,523 daily log returns and takes one step of its process model per observation, with a single normal draw. Each step between resamples is very cheap, so fixed per-observation costs such as resampling can dominate.

R and Pypomp run the same workload in a given benchmark. Each benchmark runs 36 IF2 searches from the same starting points and then runs replicate particle filters at each search's IF2 estimate, with as many particles per filter as per search. The cholera and both measles benchmarks use 5,000 particles, 100 IF2 iterations, and 36 particle filter replicates, and the S&P 500 benchmark uses 1,000 particles, 200 iterations, and 24 replicates.

Table 2 extends Table 1 with particle filter times and Pypomp on the same 36-core CPU node as R, which splits the GPU's advantage into a software part and a hardware part. On the same CPU node, Pypomp is 2.6 to 5.6 times faster than R, and moving Pypomp to the GPU makes it another 1.4 to 33 times faster. These workloads are sized for R and too small to saturate the GPU, so the hardware and total ratios understate what the GPU can do. At its peak throughput in the scaling tests (Appendix B), one GPU runs IF2 90 to 241 times as fast as R on the 36-core node.

Appendix B. GPU scaling tests

The benchmarks in Table 1 and Table 2 use the R configuration, which does not give the GPU enough work to reach its highest throughput for every model. The scaling tests in quant ran IF2 on one RTX PRO 6000 Blackwell Server Edition GPU with more particles in flight, added in two ways: more searches of 1,000 particles each, or more particles for each of a fixed number of searches. Each configuration ran in its own process and ran a short IF2

Table 2: Wall-clock time in seconds for IF2 and for particle filter (PF) evaluation, with the workloads described above. Pypomp IF2 and PF times include JIT compilation. *`panelPomp` for the 4-town panel. Software is the `pomp` time divided by the Pypomp time on the same 36 cores, hardware is the Pypomp CPU time divided by the Pypomp GPU time, and total is their product, the `pomp` time divided by the Pypomp GPU time. These workloads do not saturate the GPU, so the hardware and total ratios understate its throughput; peak cores matched in Table 1 gives the GPU's highest measured IF2 throughput. The 4-town panel has no Pypomp CPU run.

		`pomp`*	Pypomp		Ratio		
Model	Task	36 cores	36 cores	1 GPU	Software	Hardware	Total
Cholera, Dhaka	IF2	1,609	390	28.3	4.1×	14×	57×
	PF	380	67.8	8.2	5.6×	8.3×	46×
Measles, London	IF2	2,923	1,071	32.2	2.7×	33×	91×
	PF	895	302	14.5	3.0×	21×	62×
Measles, 4-town panel	IF2	8,262	—	98.2	—	—	84×
	PF	2,252	—	33.7	—	—	67×
S&P 500	IF2	703	275	34.8	2.6×	7.9×	20×
	PF	36.4	7.4	5.2	4.9×	1.4×	7.0×

twice, so that the second run excludes JIT compilation. We project each configuration's time to the benchmark's number of IF2 iterations, including compilation, and divide the benchmark's R time by it to get the number of CPU cores running R that one GPU matches. This assumes that R's time grows linearly with the number of particles and searches. The highest of these values is the Peak column of Table 1. JAX's memory preallocation was turned off, so the peak GPU memory is the memory each configuration used. No scaling test used more than 1.8 GB of GPU memory, the peak for the cholera model with 2.4 million particles.

Figure 1 shows the results. For every model, throughput peaks at between 1.0 and 2.0 million particles and falls or plateaus at larger sizes. The peak is 1.6 times the benchmark for the cholera model, 1.7 times for the London measles model, 1.3 times for the 4-town panel, and 11.9 times for the S&P 500 model. The S&P 500 model gains the most because its benchmark, with cheap steps and few particles, leaves the GPU mostly idle. For this model, adding searches beats adding particles per search at a given total number of particles by an average of 6%. Consequently, there is not a large difference in speed between adding searches or particles per search on the GPU in terms of scaling with total particles.

Appendix C. Random variable samplers

`jax.random` draws Poisson, binomial, and gamma random variables with rejection samplers, which loop a random number of times per draw. This suits GPUs poorly, since threads that run together wait for the slowest one. `pypomp.random` instead provides approximate inverse-CDF samplers, which avoid rejection loops. Table 3 compares the two on 1,000,000 draws per call, with parameters spread across the values where each sampler switches algorithms,

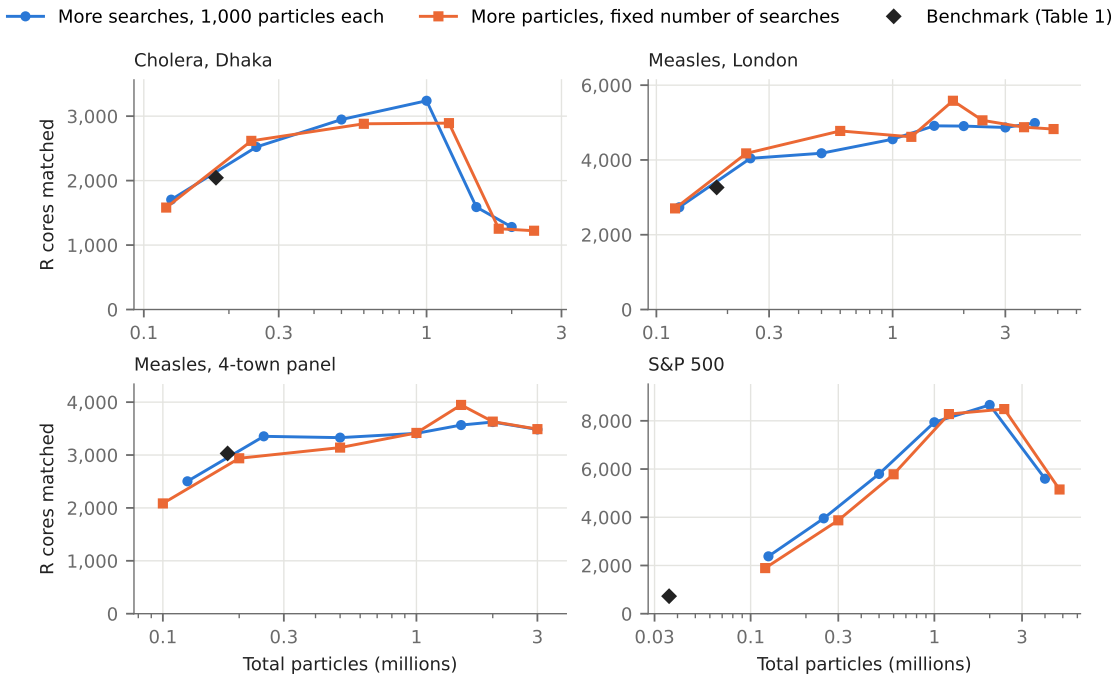


Figure 1: IF2 throughput on one GPU, as the number of CPU cores running R that the GPU matches, against the total number of particles in flight. Blue adds searches of 1,000 particles each; orange adds particles to each of 100 or 120 searches. Each configuration ran 3 or 10 IF2 iterations, projected to the benchmark's number. The x-axes use a log scale.

Table 3: Time in milliseconds for 1,000,000 draws from `pypomp.random` and `jax.random`, on one GPU and on one CPU core, with parameters spread across the values where the samplers switch algorithms. Ratio is the `jax.random` time divided by the `pypomp.random` time. `jax.random` has no negative binomial sampler. Max W_1/σ is the largest Wasserstein distance between `pypomp.random` draws and draws from `jax.random` (or `scipy.stats` for the negative binomial) over a grid of parameter values, relative to the standard deviation.

	GPU			CPU			
Distribution	`pypomp`	`jax`	Ratio	`pypomp`	`jax`	Ratio	Max W_1/σ
Poisson	0.03	1.05	35×	37.17	1,321	36×	0.010
Binomial	0.02	1.07	44×	24.56	1,349	55×	0.010
Gamma	0.03	0.76	23×	118	3,046	26×	0.011
Negative binomial	0.07	—	—	187	—	—	0.009

averaged over 20 calls. The approximation error is small: over a grid of parameter values, the Wasserstein distance between 1,000,000 draws from each `pypomp.random` sampler and from a reference sampler is at most 1.1% of the reference standard deviation. In the London measles benchmark, the `jax.random` samplers made IF2 46 times slower and particle filtering 36 times slower on the GPU.

Appendix D. England and Wales measles fit

We ran 36 MPIF searches, each with 10,000 particles for 200 iterations, and evaluated each endpoint with 36 particle filter replicates of 5,000 particles. We then combined the best estimate for each town across searches and evaluated the result with another 36 replicates. Table 4 gives the time for each stage on one GPU with Pypomp 1.0.8.

For comparison, we estimated how long the same computation would take with `panelPomp` on a 36-core CPU node. We measured `panelPomp`'s cost per particle, per IF2 iteration, and per town on the 4-town panel in Table 2, which covers the same period with the same daily time step, and scaled it linearly to the full computation. We assume `panelPomp` fits the towns jointly with MPIF, as Pypomp did, so each search runs on one core and does not parallelize across units – this would be necessary in the general case when using shared parameters. By this estimate, R would need about 4.8 months, 92 times longer than the GPU, and about 14.3 years on a single core, so the GPU delivered the throughput of about 3,300 CPU cores running R. The wall clock time can be reduced further by spreading the replication over multiple GPUs; for example, a run with the same settings but only 12 searches took one GPU 15 hours, so three GPUs running 12 searches each would finish in about 15 hours instead of 38.

Table 4: Wall-clock time for fitting the measles model to all towns in England and Wales. Pypomp times were measured on one GPU. R times are estimates for `panelPomp` on a 36-core CPU node, extrapolated from the 4-town panel in Table 2.

Stage	Pypomp (1 GPU)	R `panelPomp`, 36 cores (estimated)
IF2	35.5 hours	136.0 days
PF, all search endpoints	2.5 hours	9.3 days
PF, combined estimate	6 minutes	6.2 hours
Total	38.1 hours	145.5 days